

Chapter 6 Algorithm Test Harnesses

We cannot know which algorithm will be best for a given problem. Therefore, we need to design a test harness that we can use to evaluate different machine learning algorithms. In this tutorial, you will discover how to develop a machine learning algorithm test harness from scratch in Perl. After completing this tutorial, you will know:

- How to implement a train-test algorithm test harness.
- How to implement a k-fold cross-validation algorithm test harness. Let's get started.

6.1 Description

A test harness provides a consistent way to evaluate machine learning algorithms on a dataset. It involves 3 elements:

1. The resampling method to split-up the dataset.
2. The machine learning algorithm to evaluate.
3. The performance measure by which to evaluate predictions.

The loading and preparation of a dataset is a prerequisite step that must have been completed prior to using the test harness. The test harness must allow for different machine learning algorithms to be evaluated, whilst the dataset, resampling method and performance measures are kept constant. In this tutorial, we are going to demonstrate the test harnesses with a real dataset.

6.1.1 Pima Indians Diabetes Dataset

In this tutorial we will use the Pima Indians Diabetes Dataset. This dataset involves the prediction of the onset of diabetes within 5 years. The baseline performance on the problem is approximately 65%. You can learn more about it in Appendix A, Section A.4. Download the dataset and save it into your current working directory with the filename `pima-indians-diabetes.csv`.

6.2. Tutorial

This tutorial is broken down into two main sections:

1. Train-Test Algorithm Test Harness.
2. Cross-Validation Algorithm Test Harness. These test harnesses will give you the foundation that you need to evaluate a suite of machine learning algorithms on a given predictive modeling problem.

6.2.1 Train-Test Algorithm Test Harness

The train-test split is a simple resampling method that can be used to evaluate a machine learning algorithm. As such, it is a good starting point for developing a test harness. We can assume the prior development of a function to split a dataset into train and test sets and a function to evaluate the accuracy of a set of predictions. We need a function that can take a dataset and an algorithm and return a performance score. Below is a function named `evaluate_algorithm()` that achieves this. It takes 3 fixed arguments including the dataset, the algorithm function and the split percentage for the train-test split.

First, the dataset is split into train and test elements. Next, a copy of the test set is made and each output value is cleared by setting it to the None value to prevent the algorithm from cheating accidentally. The algorithm provided as a parameter is a function that expects the train and test datasets on which to prepare and then make predictions. The algorithm may require additional configuration parameters. This is handled by using the variable arguments %args in the evaluate_algorithm() function and passing them on to the algorithm function. The algorithm function is expected to return a list of predictions, one for each row in the training dataset. These are compared to the actual output values from the unmodified test dataset by the accuracy_metric() function. Finally, the accuracy is returned.

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip min max sum);
use sml; # Statistical Machine Learning Library
```

```
In [2]: # Defined in Section 6.2.1 Train-Test Algorithm Test Harness
# Function To Evaluate An Algorithm Using a Train/Test Split.
# Evaluate an algorithm using a train/test split
sub evaluate_algorithm_train_test_split{
    my ($self, $dataset, $algorithm, %args) = (splice(@_, 0, 3),
                                                split => 0.6,
                                                metric => undef, @_);

    my ($train, $test) = sml->train_test_split($dataset, split=>$args{split});
    my ($actual, $predicted, $score);

    # Copia profunda compacta y mapeo de la última columna
    $test = [ map { [ @$_ ] } @$test ];
    $actual = [ map { $_->[-1] } @$test ];

    $predicted = $algorithm->('sml', $train, $test, %args);
    $actual = [map {$_->[-1]} @$test];

    # Regression : Classification
    if (defined $args{metric}){
        if ($args{metric} =~ /accuracy/i) {
            $score = sml->accuracy_metric($actual, $predicted);
        }elsif($args{metric} =~ /rmse/i){
            $score = sml->rmse_metric($actual, $predicted);
        }
    }else{
        $score = (grep { $_ =~ /\d+\.\d+/ } @$actual) ?
            sml->rmse_metric($actual, $predicted) :
            sml->accuracy_metric($actual, $predicted);
    }

    return wantarray ? ($score, $train, $test, $actual, $predicted) : $score;
}

sml->add_to_class('evaluate_algorithm_train_test_split', \&{'evaluate_algorithm_train_
```

```
Out[2]: *sml::evaluate_algorithm_train_test_split
```

The evaluation function does make some strong assumptions, but they can easily be changed if needed. Specifically, it assumes that the last row in the dataset is always the output value. A different column could be used. The use of the accuracy_metric() assumes that the problem is a classification problem, but this could be changed to mean squared error for regression problems.

Let's piece this together with a worked example. We will use the Pima Indians Diabetes dataset and evaluate the Zero Rule algorithm.

The dataset was split into 60% for training the model and 40% for evaluating it. Notice how the name of the Zero Rule algorithm **zero_rule_algorithm_classification** was passed as an argument to the `evaluate_algorithm()` function. You can see how this test harness may be used again and again with different algorithms. Running the example above prints out the accuracy of the model.

```
In [3]: # Example of Train/Test Algorithm Test Harness on the Diabetes Dataset.

# Test the train/test harness
srand(1);
# load and prepare data
my $filename = '../data/pima-indians-diabetes.csv';
my $dataset = sml->load_csv($filename);

# evaluate algorithm
my $split = 0.6;
my ($accuracy, $train, $test, $actual, $predicted) =
  sml->evaluate_algorithm_train_test_split($dataset,
    \&{'sml::zero_rule_algorithm_classification'},
    split => $split,
    metric => 'accuracy');

printf 'Accuracy: %0.2f%%', $accuracy;
# Example Output From Using the Train/Test Split.
# Accuracy: 64.500%
```

Accuracy: 65.58%

Out[3]: 1

```
In [4]: my ($unique, $matrix) = sml->confusion_matrix($actual, $predicted);
sml->print_confusion_matrix($unique, $matrix);
# A/P 0 1
# 0| 202 0
# 1| 106 0
```

```
A/P 0 1
0| 202 0
1| 106 0
```

Out[4]: 1

6.2.2 Cross-Validation Algorithm Test Harness

Cross-validation is a resampling technique that provides more reliable estimates of algorithm performance on unseen data. It requires the creation and evaluation of k models on different subsets of your data, and as such is more computationally expensive. Nevertheless, it is the gold standard for evaluating machine learning algorithms. As in the previous section, we need to create a function that ties together the resampling method, the evaluation of the algorithm on the dataset and the performance calculation method. Unlike above, the algorithm must be evaluated on different subsets of the dataset many times. This means we need additional loops within our `evaluate_algorithm()` function. Below is a function that implements algorithm evaluation with cross-validation. First, the dataset is split into n folds groups called folds. Next, we loop giving each fold an opportunity to be held out of training and used to evaluate the algorithm. A copy of the list of folds is created and the held out fold is removed from this list. Then the list of folds is flattened into one long list of rows to match the algorithms expectation of a training

dataset. This is done by using a derreferentiation of an array inside the map() function. Once the training dataset is prepared the rest of the function within this loop is as above. A copy of the test dataset (the fold) is made and the output values are cleared to avoid accidental cheating by algorithms. The algorithm is prepared on the train dataset and makes predictions on the test dataset. The predictions are evaluated and stored in a list. Unlike the train-test algorithm test harness, a list of scores is returned, one for each cross-validation fold.

```
In [5]: # Function To Evaluate An Algorithm Using k-fold Cross-Validation.
# Evaluate an algorithm using a cross-validation split
sub evaluate_algorithm_cross_validation_split {
  my ($self, $dataset, $algorithm, %args) = ((splice @_, 0, 3),
                                              n_folds=>10,
                                              metric=>undef, @_);

  my $folds = sml->cross_validation_split($dataset, n_folds => $args{n_folds});

  my (@scores, @train_losses, @test_losses, @predictions, @actuals);

  foreach my $i (0 .. $$folds){

    # 1. Crear el train_folds con todos los folds excepto el actual ($i)
    my @train_folds = @$folds;
    my $test_fold = splice @train_folds, $i, 1;

    # 2. Bajamos la 3ra dimensión y concatenando todas las filas
    my $train_set = [ map { @$_ } @train_folds ];

    # 3. Copia profunda inmediata y limpia del fold de prueba
    my $test_set = [ map { [ @$_ ] } @$test_fold ];

    # 4. Ejecutar el algoritmo pasándole las referencias estructuradas
    my ($predicted, $train_loss, $test_loss) =
      $algorithm->('sml', $train_set, $test_set, %args);

    # 5. Extraer las etiquetas reales
    my $actual = [ map { $_->[-1] } @$test_fold ];

    # Regression : Classification
    if (defined $args{metric}){
      if ($args{metric} =~ /accuracy/i) {
        push @scores, sml->accuracy_metric($actual, $predicted);
      }elseif($args{metric} =~ /rmse/i){
        push @scores, sml->rmse_metric($actual, $predicted);
      }
    }else{
      push @scores, (grep { $_ =~ /\d+\.\d+/ } @$actual) ?
        sml->rmse_metric($actual, $predicted) :
        sml->accuracy_metric($actual, $predicted);
    }

    # Guardar métricas/estados del fold actual si es necesario (el código original se
    push @train_losses, $train_loss;
    push @test_losses, $test_loss;
    push @actuals, $actual;
    push @predictions, $predicted;
  }

  # Retornar los acumulados estructurados según necesites
  return wantarray ? (\@scores, \@train_losses, \@test_losses, \@actuals, \@prediction
}
```

```
sml->add_to_class('evaluate_algorithm_cross_validation_split', \&{'evaluate_algorithm_
```

Out[5]: *sml::evaluate_algorithm_cross_validation_split

Although slightly more complex in code and slower to run, this function provides a more robust estimate of algorithm performance. We can tie all of this together with a complete example on the diabetes dataset with the Zero Rule algorithm.

A total of 5 cross-validation folds were used to evaluate the Zero Rule Algorithm. As such, 5 scores were returned from the evaluate_algorithm() algorithm. Running this example both prints these list of scores calculated and prints the mean score.

```
In [6]: # Example of the k-fold Cross-Validation Algorithm Test Harness on the Diabetes Dataset
# Test cross validation test harness
srand(1);
# load and prepare data
$filename = '../data/pima-indians-diabetes.csv';
$dataset = sml->load_csv($filename);

# evaluate algorithm
my $n_folds = 5;
my ($scores, $train_losses, $test_losses, $actuals, $predictions) =
    sml->evaluate_algorithm_cross_validation_split($dataset,
        \&{'sml::zero_rule_algorithm_classification'},
        n_folds => $n_folds,
        metric => 'accuracy');

printf "Scores: %s\n", dump @$scores;
printf "Mean Accuracy: %0.2f%%", sum(@$scores) / @$scores;

# Example Output From Using the Cross-Validation Test Harness.
# Scores: (62.7, 68.6, 62.7, "68.0", 63.4)
# Mean Accuracy: 65.080%
```

Scores: (62.75, 68.63, 62.75, 67.97, "63.40")

Mean Accuracy: 65.10%

Out[6]: 1

```
In [7]: for my $item (zip $scores, $actuals, $predictions){
    my ($accuracy, $actual, $predicted) = @$item;
    my ($unique, $matrix) = sml->confusion_matrix($actual, $predicted);
    printf "Accuracy: %0.2f%%\n", $accuracy;
    sml->print_confusion_matrix($unique, $matrix);
}
# Accuracy: 62.75%
# A/P 0 1
# 0| 96 0
# 1| 57 0
# Accuracy: 68.63%
# A/P 0 1
# 0| 105 0
# 1| 48 0
# Accuracy: 62.75%
# A/P 0 1
# 0| 96 0
# 1| 57 0
# Accuracy: 67.97%
# A/P 0 1
# 0| 104 0
```

```
# 1| 49 0
# Accuracy: 63.40%
# A/P 0 1
# 0| 97 0
# 1| 56 0
```

```
Accuracy: 62.75%
A/P 0 1
0| 96 0
1| 57 0
```

```
Accuracy: 68.63%
A/P 0 1
0| 105 0
1| 48 0
```

```
Accuracy: 62.75%
A/P 0 1
0| 96 0
1| 57 0
```

```
Accuracy: 67.97%
A/P 0 1
0| 104 0
1| 49 0
```

```
Accuracy: 63.40%
A/P 0 1
0| 97 0
1| 56 0
```

You now have two different test harnesses that you can use to evaluate your own machine learning algorithms.

6.3 Extensions

This section lists extensions to this tutorial that you may wish to consider.

- **Parameterized Evaluation.** Pass in the function used to evaluate predictions, allowing you to seamlessly work with regression problems.
- **Parameterized Resampling.** Pass in the function used to calculate resampling splits, allowing you to easily switch between the train-test and cross-validation methods.
- **Standard Deviation Scores.** Calculate the standard deviation to get an idea of the spread of scores when evaluating algorithms using cross-validation.

6.4 Review

In this tutorial, you discovered how to create a test harness from scratch to evaluate your machine learning algorithms. Specifically, you now know:

- How to implement and use a train-test algorithm test harness.
- How to implement and use a cross-validation algorithm test harness.